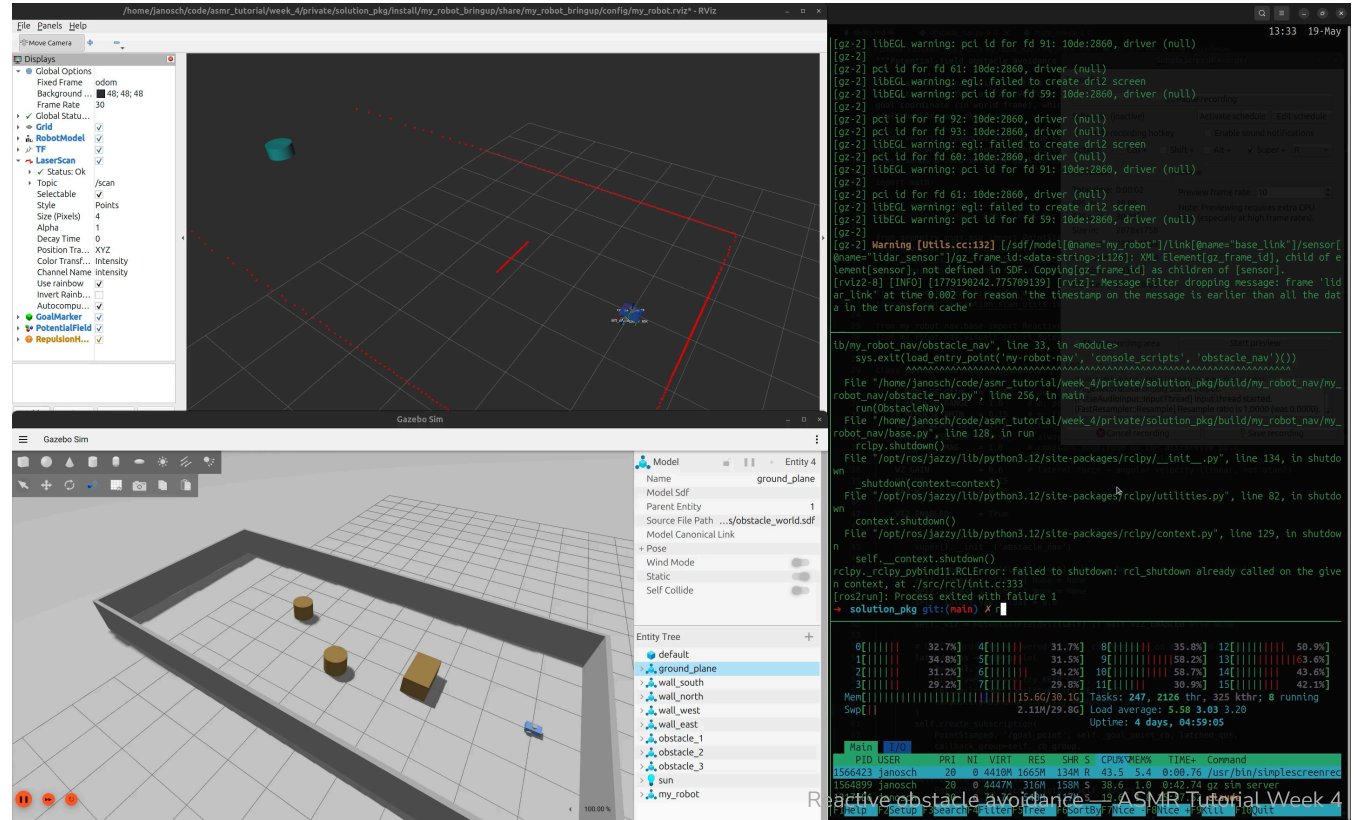


Autonomous Systems and Mobile Robots

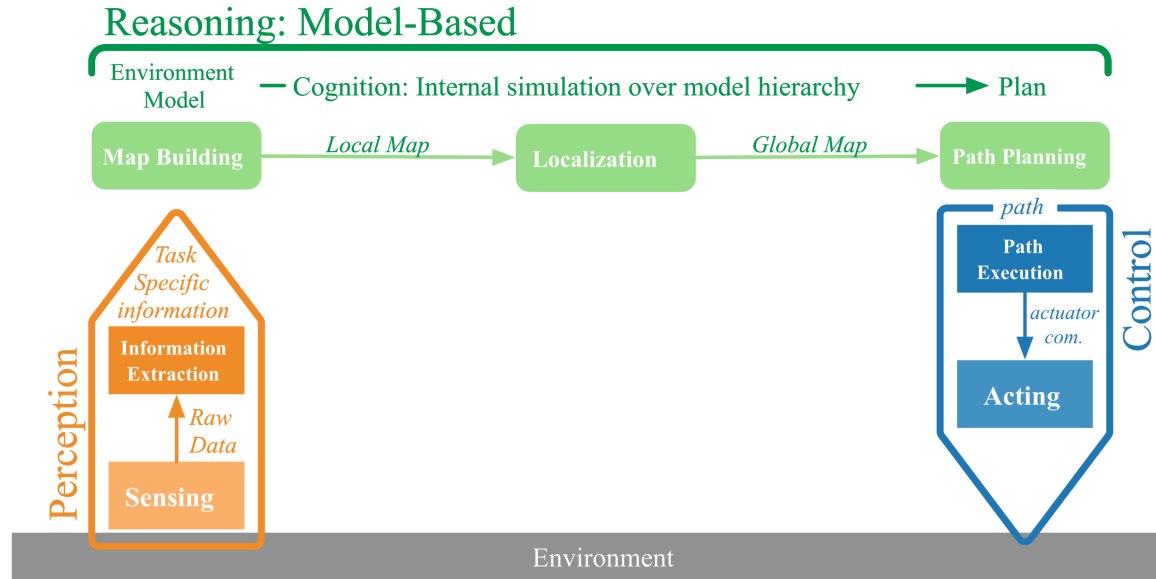
MiniLab 1 — Reactive Obstacle Navigation



Recap

Where you are, where we go today

Perceive · Reason · Act



Perceive

LiDAR integration, scan indexing, frame transforms. Done in Weeks 2–3.

Reason + Act

Turn sensor data into a motion decision. Issue a `SetVelocity` goal. Today's focus.

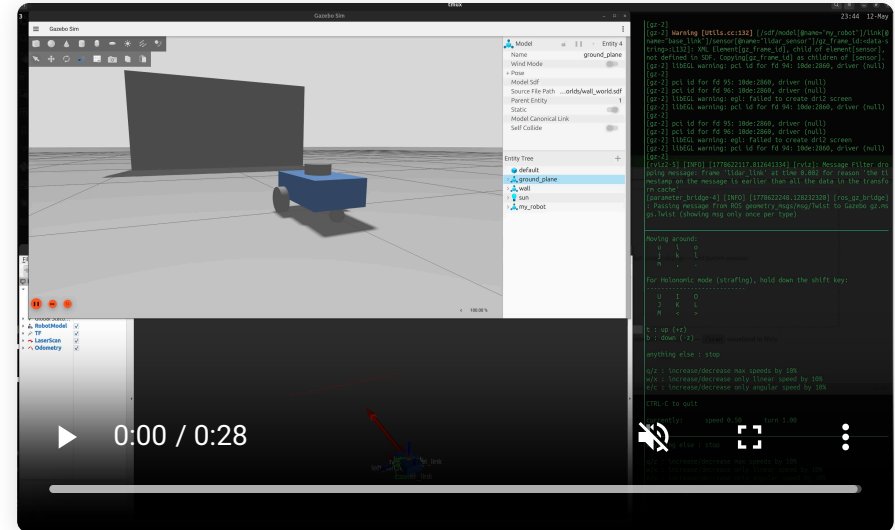
Semester at a Glance

#	Date	Theme
0	14.04	Python Intro
1	21.04	ROS2 Packaging + PID Control
2	05.05	Robot Body and Blind Locomotion
3	12.05	Exteroception — LiDAR
4	19.05	MiniLab 1 Briefing ← you are here
—	26.05	Pfingsten — no session
5	02.06	Troubleshooting
6	09.06	FK/IK Robotic Manipulator
7	16.06	MiniLab 2 Briefing
8	23.06	Troubleshooting
9	30.06	Recap

Last Week

Week 3 — Exteroception: LiDAR

- `sensor_msgs/LaserScan` — message format, ranges array, angle parameters
- Forward-index formula: bearing $\rightarrow$ array index
- TF2 frame transforms: `lidar_link` $\rightarrow$ `base_link`
- Systematic vs stochastic sensor error



Today's Content

Reactive Control

Closing the perceive-reason-act loop

A typical navigation stack

What the robot has, what's missing

Reactive approaches

Stacking reactive rules

MiniLab 1 — Briefing

Starting point (existing, provided, what you build)

The task: Reactive Obstacle Navigation

`goal_checker_node` and `/goal_reached`

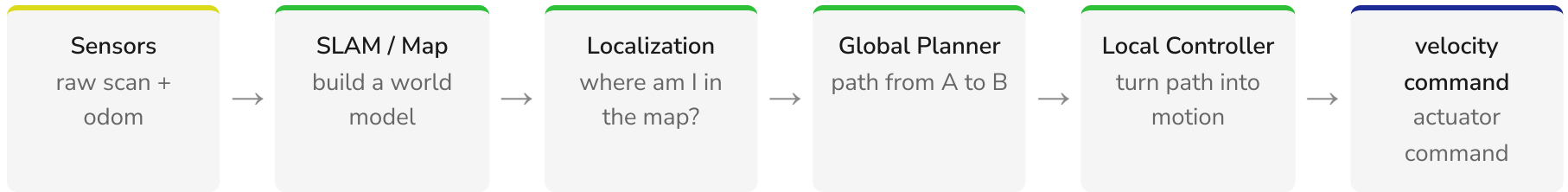
Solution preview

Reactive Control

From textbook stack to one reactive layer

How most robots navigate

A typical navigation stack includes the following



The classical Sense-Plan-Act pipeline. A world model lets a planner find a path; the controller executes it. Most production robots do this.

Status quo - Your robot, your data, your constraints

Perceive

`/scan` — `LaserScan` , 360-sample ring (W3)

`/odom` — `Odometry` , pose + velocity (W2)

`/scan_points` — `PointCloud2` , cartesian neighbours (opt-in, W4)

Act

`SetVelocity` action goal → `velocity_controller_node` (provided)

`/cmd_vel` owned by the controller

Tools (provided)

`sector_min` , `get_position` , `get_yaw` (`my_robot_perception`)

`goal_checker_node` → `/goal_reached`

Constraints:

No map of the world → no localization in a map → no global planner

Only the goal coordinate, not a path → no waypoints to follow

Single-rate sensor callback → you act on what you sense *now*

Reactive Control

Reactive Control: bind sensor events directly to commands — no map, no plan. Stack such rules in priority order.

Approach	Uses	Idea
Forward threshold	<code>/scan</code>	Closest thing in front decides whether to drive or turn
Sector comparison	<code>/scan</code>	Steer toward the side with more space
Wall-following	<code>/scan</code>	Keep one wall at a target distance — the right-hand rule
Stuck detection	<code>/odom</code> , <code>/scan</code>	No displacement → recover with a deliberate turn

You've used `/scan` indexing in Week 3 Task 4 and `/odom` in Week 2 — that's all you need.

Stacking Reactive Rules

Each rule is one Reason→Act binding. Stack them in priority order.

Layer	Rule
Safety	<code>if front < 0.3 m → stop</code>
Avoidance	<code>if front < 0.6 m → turn away</code>
Navigation	<code>else → drive towards goal</code>

Higher-priority rules inhibit lower ones. Start with avoidance; add safety or navigation as separate rules if needed.

ROS 2 Communication

What happens when you call `publish()`

Topics are network streams

```
scan = LaserScan( ... )  
self.pub.publish(scan)
```



Underneath that one line:

- Message serialised to bytes
- DDS middleware finds matching subscribers (same machine or remote)
- Bytes travel over the network
- Subscriber deserialises → callback fires

Topics are network streams. The network has knobs — that is what QoS configures.

Quality of Service

QoS controls how the communication stack handles your stream. Two parameters matter for the MiniLab.

"Is every message critical, or only the latest?"

Reliability

- **Reliable** — every message; the stack retries until the subscriber acknowledges.
- **Best effort** — drops messages if the network is busy; latency wins over completeness.

"Constant stream, or rare announcement?"

Durability

- **Transient local** — rare announcement; the publisher keeps the last message and replays it to subscribers that connect later (latched).
- **Volatile** — continuous stream; live messages only, no replay.

Publisher and subscriber must be compatible. If they are not, no messages flow — and no error is raised.

In your starter package

`/scan` — **best-effort**

Published with `qos_profile_sensor_data`. A default (reliable) subscription doesn't connect.

```
from rclpy.qos import qos_profile_sensor_data

self.create_subscription(
    msg_type=LaserScan,
    topic='/scan',
    callback=cb,
    qos_profile=qos_profile_sensor_data,
)
```

`/goal_point`, `/goal_reached` — **transient_local**

Latched, published once at startup. A default (volatile) subscription connects but the latched message is invisible.

```
from rclpy.qos import QoSProfile, DurabilityPolicy

latched = QoSProfile(
    depth=1,
    durability=DurabilityPolicy.TRANSIENT_LOCAL,
)

self.create_subscription(
    msg_type=PointStamped,
    topic='/goal_point',
    callback=cb,
    qos_profile=latched,
)
```

Silent failure is the norm. If your callback isn't firing, suspect QoS first.

MiniLab 1

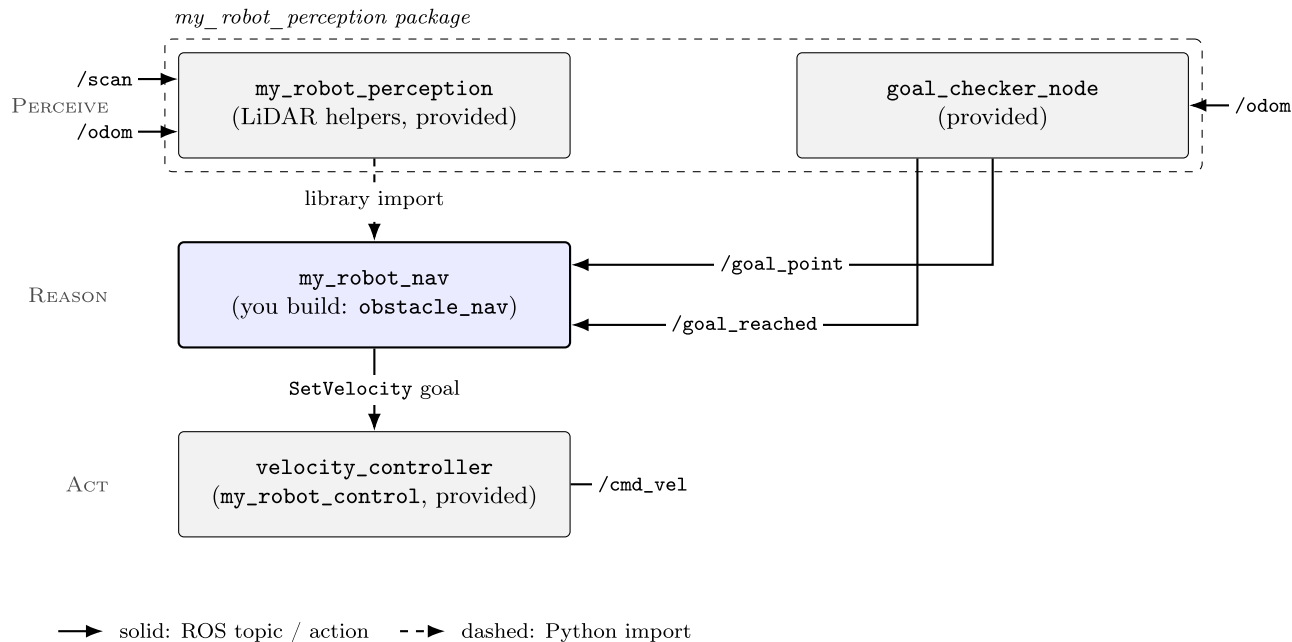
Reactive obstacle navigation

Starting Point

Package	Role	Origin
<code>my_robot_description</code>	URDF: body, wheels, LiDAR	Week 2 + 3
<code>my_robot_bringup</code>	launch files, <code>bridge.yaml</code> , RViz config, world SDFs	Week 2 + 3
<code>my_robot_control</code>	open-loop helpers, <code>velocity_controller_node</code>	Week 2 + this week
<code>my_robot_perception</code>	sensor helpers, <code>goal_checker_node</code> , <code>scan_to_pointcloud_node</code>	provided this week
<code>my_robot_interfaces</code>	action definitions (<code>SetVelocity.action</code>)	provided this week

You create one new package: `my_robot_nav` .

Architecture



Perception helpers feed your nav node; your nav node sends `SetVelocity` goals; the controller owns `/cmd_vel`. `/goal_reached` is an event from the goal-checker.

MiniLab 1 — at a glance

Due	10 June 2026, 23:59
Groups	2–3 students (pre-assigned)
Total points	40 — pass at 20
Submission	ZIP of full workspace → LearnWeb

Package builds cleanly; deps declared · 5 pts

`sim.launch.py` brings up world + `goal_checker_node` + `velocity_controller_node` · 5 pts

Subscribes `/goal_point`, `/scan`, `/odom`; commands via `SetVelocity` action (no direct `/cmd_vel`) · 5 pts

Steering visibly depends on LiDAR data, per the validity rule · 5 pts

Named viz/debug signal implemented, observable when node runs, noted in README · 5 pts

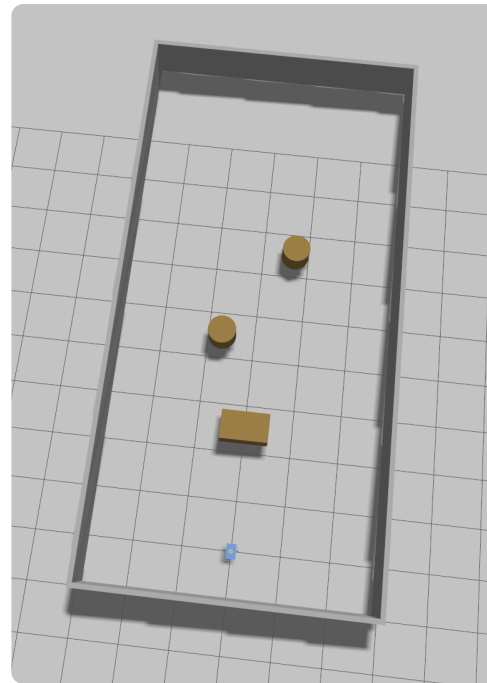
No visible collisions during the run · 5 pts

Reaches the goal region; clean stop on `/goal_reached` (final zero-velocity goal sent on arrival) · 10 pts

Validity rule. Your nav node's velocity commands must be driven by LiDAR data, and motion must go through the `SetVelocity` action client — your node does not publish `/cmd_vel` directly. A submission that violates this rule (ignores LiDAR or bypasses the action client) is rejected regardless of whether the robot arrives.

Reactive Obstacle Navigation

- Implement `obstacle_nav` : drive past the obstacles to the goal
- Reactive logic — use the sensor streams that make sense for your approach
- Command velocity through the `SetVelocity` action
- Publish or log a named viz/debug signal of your choice (note it in your README)

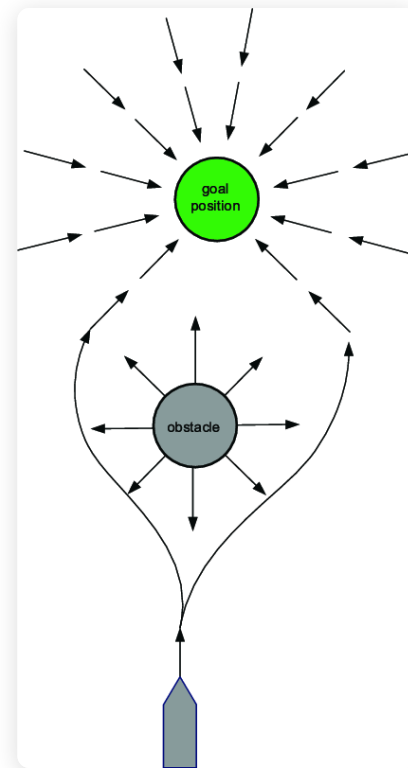


One box and two cylinders between spawn and goal.

My approach (reference)

- Potential field on `/scan_points` (cartesian LiDAR points).
- **Attractive:** unit vector from robot to goal in world frame, rotated into `base_link`.
- **Repulsive:** per-point falloff + *vortex tangent* to slip around obstacles (avoids local-minimum oscillation).
- Force $\rightarrow$ twist: $v_x \propto F.x$, $w_z \propto F.y$.
- Live viz: arrows + heatmap on RViz — debug visually, catch frame/sign bugs.

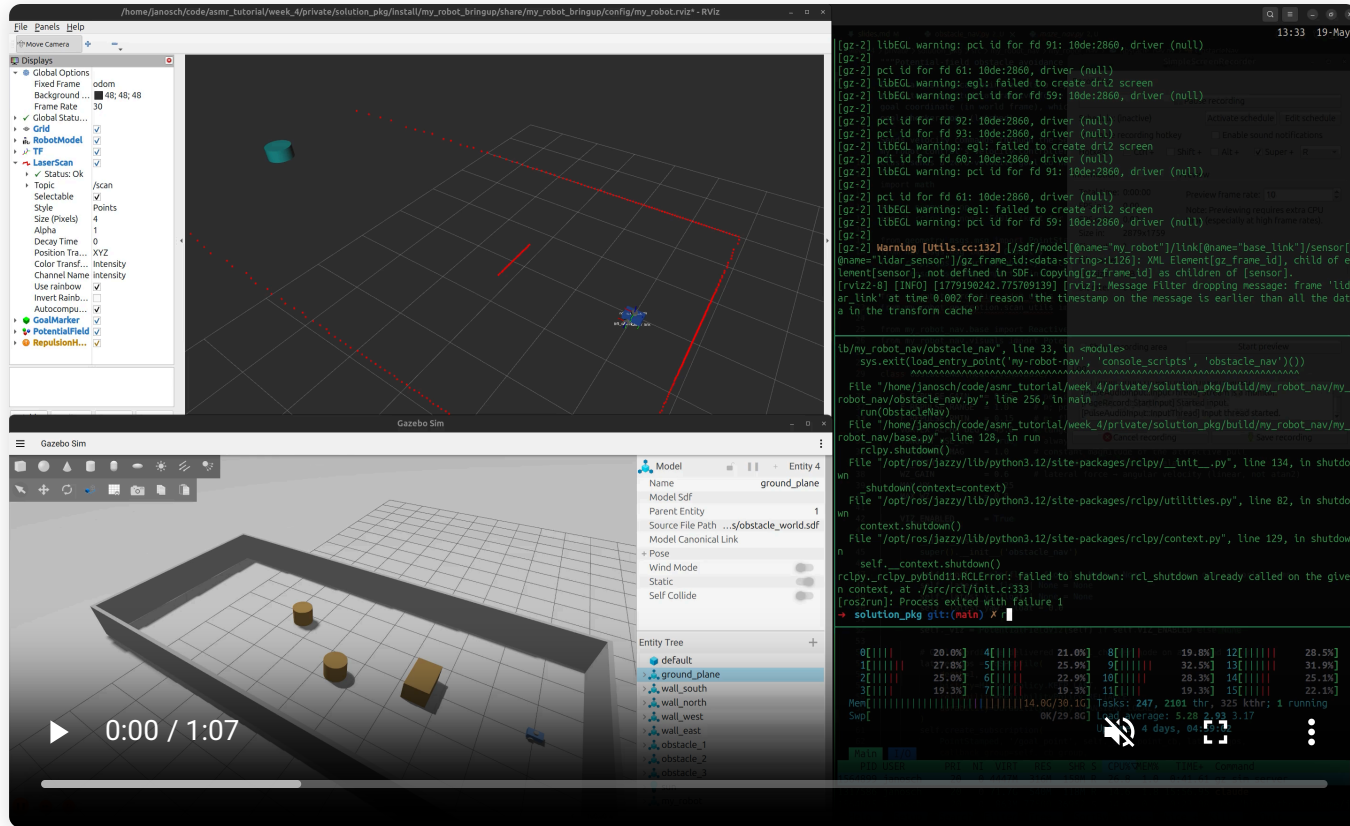
One approach — not the only one. Pick what you can explain and implement cleanly.



Concept of an artificial potential field: goal pulls, obstacles push.

Image: [ResearchGate publication 347771430](#).

Solution Preview



Get started

1. Form your group (2–3 students).
2. Read the sheet, copy `reference_ws/src/` into your workspace, and explore the provided packages (`my_robot_perception` helpers, `velocity_controller_node` , `goal_checker_node`).
3. Research your approach - Plan before you code.